Fatec Jales Prof. José Camargo | Centro Paula Souza | GOVERNO DO ESTADO DE SÃO PAULO

Faculdade de Tecnologia Professor José Camargo

Sprint Documentation		
Sprint 17	**Start Date: 05/05/2026**	**Final Date: 08/05/2026**
Project Name:	**civitas-backend**	
Ano: 5º	**Análise e Desenvolvimento de Sistemas - AMS**	

Team Members
Felipe Pacheco Bianchini
Wendell Alves Nascimento
Gustavo Henrique Moreira Porto
Giovanni da Silva Nascimento

Sprint Backlog			
Task #	Description	Start Date	Final date
#96	Implementação de Campos Opcionais Dinâmicos com JSONB	05/05/2026	08/05/2026

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#96	Implementar suporte a campos opcionais dinâmicos nas entidades TipoDespesa e Despesa. A funcionalidade deverá permitir: Definir campos adicionais em TipoDespesa Armazenar valores opcionais desses campos em Despesa Flexibilizar o cadastro sem necessidade de alteração estrutural no banco Ajustar DTOs, Services, Mapeamentos e Endpoints para suportar os novos campos Garantir integridade entre os campos configurados e os valores preenchidos	Felipe Pacheco Bianchini, Wendell Alves Nascimento , Gustavo Henrique Moreira Porto, Giovanni da Silva Nascimento	Completo	5 horas	5 horas

Sprint Description

Tarefa: Implementação de Campos Opcionais Dinâmicos com JSONB

Objetivo

Permitir que os tipos de despesa possuam campos adicionais configuráveis dinamicamente utilizando JSONB, possibilitando que cada despesa armazene valores opcionais personalizados conforme o tipo relacionado.

A implementação deverá manter compatibilidade com a estrutura atual do sistema e adaptar os endpoints existentes para suportar o novo comportamento.

Descrição

Implementar suporte a campos opcionais dinâmicos nas entidades TipoDespesa e Despesa.

A funcionalidade deverá permitir:

- Definir campos adicionais em TipoDespesa
- Armazenar valores opcionais desses campos em Despesa
- Flexibilizar o cadastro sem necessidade de alteração estrutural no banco
- Ajustar DTOs, Services, Mapeamentos e Endpoints para suportar os novos campos
- Garantir integridade entre os campos configurados e os valores preenchidos

Critérios de Aceite

Alterações na Entidade TipoDespesa

- Adicionar campo JSONB para armazenar definição de campos opcionais

O campo deverá armazenar apenas a estrutura/configuração dos campos adicionais

Permitir armazenamento de lista de campos opcionais configuráveis

Exemplo:

```
{
 "camposOpcionais": [
  "numeroNota",
  "fornecedor",
  "centroCusto"
 ]
}
```

Alterações na Entidade Despesa

- Adicionar campo JSONB para armazenar valores dos campos opcionais

Permitir preenchimento parcial ou total dos campos opcionais

Não obrigar preenchimento de todos os campos

Permitir valores nulos quando aplicável

Exemplo:

```
{
 "numeroNota": "12345",
 "fornecedor": "Papelaria Central",
 "centroCusto": null
}
```

Alterações nos DTOs

- Adicionar os novos campos JSONB nos DTOs de TipoDespesa

Adicionar os novos campos JSONB nos DTOs de Despesa

Garantir serialização e desserialização correta dos objetos JSON

Ajustar validações dos DTOs conforme necessidade

Alterações nos Endpoints de TipoDespesa

-
- Ajustar endpoint de criação (POST)
    - Permitir envio dos campos opcionais
- Ajustar endpoint de atualização (PUT)
    - Permitir alteração da estrutura dos campos opcionais
- Ajustar endpoint de listagem/busca (GET)
    - Retornar os campos opcionais cadastrados

Alterações nos Endpoints de Despesa

-
- Ajustar endpoint de criação (POST)
    - Permitir envio dos valores opcionais
- Ajustar endpoint de atualização (PUT)
    - Permitir atualização parcial ou total dos campos opcionais
- Ajustar endpoint de listagem/busca (GET)

- Retornar os valores opcionais associados à despesa

Alterações na Camada Service

- Implementar validações relacionadas aos campos opcionais

Garantir integridade entre os campos definidos em TipoDespesa e os valores enviados em Despesa

Validar estrutura JSON antes da persistência

Centralizar regras de negócio relacionadas aos novos campos

Alterações em Mapeamentos

- Ajustar mapeamentos entre Model e DTO

Garantir persistência correta dos campos JSONB

Revisar conversões e serializações utilizadas atualmente

Validações

- Validar se os campos enviados em Despesa existem na configuração do TipoDespesa relacionado à despesa, considerando o relacionamento atualmente utilizado pelo sistema

Não permitir campos desconhecidos

Garantir compatibilidade estrutural entre TipoDespesa e Despesa

Validar estrutura JSON antes de persistir

Tratar valores nulos corretamente

Validar payloads inválidos ou malformados

Garantir retorno de mensagens claras ao usuário

Banco de Dados

- Criar migration adicionando colunas JSONB

Garantir compatibilidade com PostgreSQL

Definir valores padrão quando necessário

Garantir retrocompatibilidade com registros antigos

Normalização e Boas Práticas

- Implementar validações na camada SERVICE

Não depender exclusivamente de DataAnnotations

Validar entradas antes da persistência

Padronizar nomenclatura dos campos JSON

Revisar tratamento de nullables

Validar cenários de atualização parcial

Revisar impacto nos retornos da API e documentação Swagger

Verificar funcionamento completo antes do envio da task

Observações e Sugestões

- Utilizar JSONB para melhor performance e flexibilidade no PostgreSQL
- Evitar armazenar dados fixos dentro do JSONB
- A implementação deverá respeitar a estrutura atual do sistema e seus relacionamentos
- Revisar impacto em filtros, paginações e consultas existentes
- Garantir que os campos adicionais não afetem funcionalidades já implementadas

Sprint Results

Sprint 17 — Campos Opcionais Dinâmicos com JSONB

Nesta sprint foi implementado o suporte a campos opcionais dinâmicos para os cadastros de TipoDespesa e Despesa. A funcionalidade permite que cada tipo de despesa declare uma lista de campos adicionais configuráveis, enquanto cada despesa pode armazenar os respectivos valores preenchidos. Esses dados são persistidos em colunas do tipo JSONB no PostgreSQL, evitando a necessidade de alterações estruturais no banco sempre que novos campos forem necessários.

Foram adicionadas as propriedades CamposOpcionais na entidade TipoDespesa e ValoresOpcionais na entidade Despesa. Também foram ajustados os DTOs correspondentes para expor esses dados de forma mais adequada à API, utilizando lista de strings para os campos declarados e dicionário de chave/valor para os valores preenchidos.

A implementação também incluiu um helper específico, CamposOpcionaisJsonHelper, responsável por centralizar a lógica de parsing, serialização e validação dos dados em JSON. Além disso, o AutoMapper foi configurado para converter automaticamente os dados entre Model e DTO, utilizando leitura defensiva para evitar que registros corrompidos derrubem consultas da API.

Na camada de serviço, foram adicionadas validações para garantir a integridade dos dados. Em TipoDespesaService, os campos opcionais declarados passam por validações de duplicidade, limite de quantidade e tamanho máximo. Em DespesaService, foi implementada a validação que garante que todos os valores opcionais informados em uma despesa estejam previamente declarados no respectivo TipoDespesa, por meio do relacionamento com UnidadeConsumidora.

Também foi criado um script SQL standalone, localizado em Civitas.WebAPI/sql/add_campos_opcionais_jsonb.sql, responsável por adicionar as colunas

camposopcionais e valoresopcionais às tabelas correspondentes. O script é idempotente, utiliza ADD COLUMN IF NOT EXISTS e mantém retrocompatibilidade por permitir valores nulos.

A entrega foi validada por testes automatizados e verificações funcionais. Foram criados testes unitários para o helper de campos opcionais, cobrindo parsing, serialização, duplicidade, valores inválidos e validação de chaves desconhecidas. Também foram adicionados testes específicos para a validação dos valores opcionais no serviço de despesas. Além disso, foram executados builds da WebAPI e da solução completa, ambos finalizados sem erros.

Por fim, foram realizados testes funcionais em ambiente local com PostgreSQL em Docker, aplicação executando em modo Development e requisições autenticadas via curl. Os testes confirmaram o cadastro, consulta, atualização e validação dos campos opcionais em TipoDespesa e Despesa, incluindo cenários de sucesso e erro.

Observações Técnicas

Não foram criadas migrations EF versionadas, pois o repositório possui política de não versionamento da pasta Migrations. Por esse motivo, a alteração de schema foi entregue por meio de script SQL standalone. Também não foram adicionadas novas dependências ao projeto, mantendo a implementação compatível com a estrutura atual da aplicação.

+ TipoDespesa agora possui CamposOpcionais

+ Despesa agora possui ValoresOpcionais

+ DTOs ajustados para expor os campos dinâmicos de forma limpa na API

+ Criado helper para tratar JSON dos campos opcionais

+ AutoMapper atualizado para converter JSON entre Model e DTO

+ TipoDespesaService valida estrutura dos campos opcionais

+ DespesaService valida se os valores enviados pertencem aos campos declarados

+ Criado script SQL para adicionar colunas JSONB no PostgreSQL

+ Criados testes unitários para helper e validação de despesa

+ Adicionado InternalsVisibleTo para suporte aos testes

- Não foram criadas migrations EF versionadas

- Não foram adicionadas novas dependências ao projeto

Assinaturas